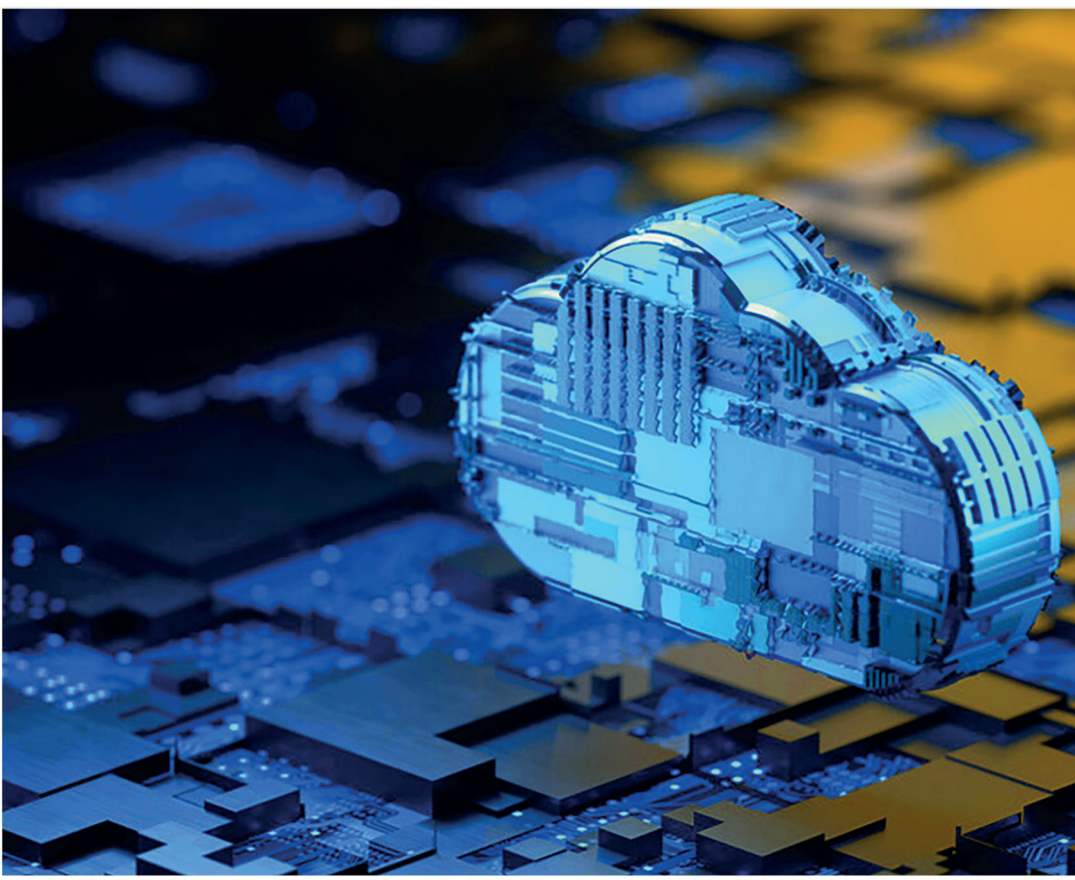
The Institution of Engineering and Technology

Managing Internet of Things Applications across Edge and Cloud Data Centres

Edited by
Rajiv Ranjan, Karan Mitra, Prem Prakash Jayaraman and Albert Y. Zomaya



IET COMPUTING SERIES 27

Managing Internet of Things Applications across Edge and Cloud Data Centres

IET Book Series on Big Data – Call for Authors

Editor-in-Chief: Professor Albert Y. Zomaya, University of Sydney, Australia

The topic of Big Data has emerged as a revolutionary theme that cuts across many technologies and application domains. This new book series brings together topics within the myriad research activities in many areas that analyse, compute, store, manage and transport massive amount of data, such as algorithm design, data mining and search, processor architectures, databases, infrastructure development, service and data discovery, networking and mobile computing, cloud computing, high-performance computing, privacy and security, storage and visualization.

Topics considered include (but not restricted to) IoT and Internet computing; cloud computing; peer-to-peer computing; autonomic computing; data centre computing; multi-core and many core computing; parallel, distributed and high-performance computing; scalable databases; mobile computing and sensor networking; green computing; service computing; networking infrastructures; cyberinfrastructures; e-Science; smart cities; analytics and data mining; Big Data applications and more.

Proposals for coherently integrated International co-edited or co-authored handbooks and research monographs will be considered for this book series. Each proposal will be reviewed by the editor-in-chief and some board members, with additional external reviews from independent reviewers. Please email your book proposal for the IET Book Series on Big Data to: Professor Albert Y. Zomaya at albert.zomaya@sydney.edu.au or to the IET at author_support@theiet.org.

Managing Internet of Things Applications across Edge and Cloud Data Centres

Edited by
Rajiv Ranjan, Karan Mitra, Prem Prakash Jayaraman and Albert Y. Zomaya

The Institution of Engineering and Technology

Published by The Institution of Engineering and Technology, London, United Kingdom

The Institution of Engineering and Technology is registered as a Charity in England & Wales (no. 211014) and Scotland (no. SC038698).

© The Institution of Engineering and Technology 2024

First published 2024

This publication is copyright under the Berne Convention and the Universal Copyright Convention. All rights reserved. Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Designs and Patents Act 1988, this publication may be reproduced, stored or transmitted, in any form or by any means, only with the prior permission in writing of the publishers, or in the case of reprographic reproduction in accordance with the terms of licences issued by the Copyright Licensing Agency. Enquiries concerning reproduction outside those terms should be sent to the publisher at the undermentioned address:

The Institution of Engineering and Technology
Futures Place
Kings Way, Stevenage
Hertfordshire, SG1 2UA, United Kingdom

www.theiet.org

While the authors and publisher believe that the information and guidance given in this work are correct, all parties must rely upon their own skill and judgement when making use of them. Neither the authors nor publisher assumes any liability to anyone for any loss or damage caused by any error or omission in the work, whether such an error or omission is the result of negligence or any other cause. Any and all such liability is disclaimed.

The moral rights of the authors to be identified as authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

British Library Cataloguing in Publication Data
A catalogue record for this product is available from the British Library

ISBN 978-1-78561-779-9 (hardback)
ISBN 978-1-78561-780-5 (PDF)

Typeset in India by MPS Limited
Printed in the UK by CPI Group (UK) Ltd, Croydon

Cover credit: Just Super/E+ *via Getty Images*

Contents

About the editors xiii
Foreword xv
Preface xvii

1 Task offloading using fog analytics 1
Rakesh Matam and Somanath Tripathy
1.1 Introduction 1
1.2 Fog-computing architecture 2
1.3 Task offloading 5
1.3.1 Task and task offloading 5
1.3.2 When to offload tasks? 7
1.4 Fog analytics 7
1.4.1 Learning approaches for task offloading 8
1.5 Task offloading using fog analytics 12
1.6 Discussion and conclusion 13
References 14

2 Resource management in edge and fog computing using FogBus2 framework 17
Mohammad Goudarzi, Qifan Deng and Rajkumar Buyya
2.1 Introduction 17
2.2 FogBus2 framework 19
2.2.1 Main components 19
2.2.2 Interaction scenario 22
2.2.3 Communication protocol 23
2.2.4 Main capabilities 23
2.3 Installation of FogBus2 framework 28
2.3.1 Building from scratch 28
2.3.2 Pulling from docker hub 29
2.4 Sample FogBus2 setup 30
2.4.1 P2P VPN setup 32
2.4.2 Running FogBus2 components 33
2.5 Extending FogBus2 framework and new IoT applications 35
2.5.1 Implementation of new IoT applications 35
2.5.2 Implementation of new scheduling policy 45

2.5.3 Evaluation results 49
2.6 Summary 51
References 51

3 Resource management and cloud-RAN implementation for narrowband-IoT systems 53
M. Pavan Reddy and Abhinav Kumar
3.1 Introduction 53
3.2 Related work 55
3.3 NB-IoT physical layer architecture 56
3.4 Resource management in NB-IoT 59
3.4.1 Performance metrics 59
3.4.2 Allocation algorithms 59
3.5 CRAN 66
3.5.1 NB-IoT with CRAN 67
3.5.2 Advantages 68
3.6 Standardization 68
3.6.1 Release 16 68
3.6.2 Release 17 70
3.7 Conclusion 71
References 72

4 Introduction to benchmarking IoT middleware platforms 75
Shalmoly Mondal, Prem Prakash Jayaraman, Alireza Hassani and Pari Delir Haghighi
4.1 Introduction 75
4.2 Motivating scenario 77
4.3 Middleware-based architecture of the IoT ecosystem 79
4.3.1 Component and functionalities of IoT middleware platforms 81
4.4 IoT middleware platforms – a summary 83
4.4.1 Open source IoT middleware platforms 83
4.4.2 Commercial IoT platforms 85
4.5 Benchmarking IoT middleware platforms 87
4.5.1 Introduction to benchmarking 87
4.5.2 State-of-the-art in benchmarks in IoT 89
4.6 Towards a holistic framework for benchmarking IoT middleware – challenges and open issues 91
4.7 Conclusion 92
References 93

5 SLA conceptual model for IoT applications 97
Awatif Alqahtani, Ellis Solaiman and Rajiv Ranjan
5.1 Introduction 97

5.2 An end-to-end SLA conceptual model for IoT applications 100
5.3 Vocabulary terms of the configuration parameters and QoS metrics 104
5.3.1 Infrastructure resources 104
5.3.2 Service concept 107
5.4 Evaluation 119
5.4.1 Experiment 119
5.4.2 Participants 119
5.4.3 Procedure 119
5.4.4 Experimental results 121
5.5 Conclusion and future work 121
References 121

6 SLA representation and awareness within blockchain in the context of IoT 127
Ali Alzubaidi, Karan Mitra and Ellis Solaiman
6.1 Introduction 127
6.1.1 Overview on SLA definition and negotiation 129
6.2 SLA representation in related works 130
6.2.1 SLA-agnostic approaches 130
6.2.2 SLA-aware approaches 132
6.2.3 SLA negotiation in related works 134
6.3 IRAFUTAL: proposed principles for SLA representation 135
6.4 Proposed SLA representation approach 137
6.4.1 Overview of the state storage capability 137
6.4.2 SLA as blockchain assets 138
6.4.3 Formal SLA specification 139
6.4.4 Designing an SLA data model 140
6.5 Implementation of the SLA data model 143
6.5.1 The logic of SLA asset manager 145
6.6 Evaluation and observation 152
6.6.1 Failure test units 152
6.6.2 Use case 1: SLA definition 153
6.6.3 Use case 2: SLA negotiation 154
6.6.4 Threats to validity 158
6.7 Conclusion 158
References 159

7 IoT monitoring with blockchain: generating smart contracts from service level agreements 165
Adam Booth, Awatif Alqahtani and Ellis Solaiman
7.1 Introduction 165
7.2 Background 167
7.2.1 IoT 167
7.2.2 SLAs 168

7.2.3 Blockchain 169
7.2.4 Smart contracts 169
7.2.5 Hyperledger Fabric 170
7.3 Related work 171
7.4 Design and implementation 172
7.4.1 SLA 173
7.4.2 From SLA to smart contract Java library 174
7.4.3 FromSLAToSmartContract library 174
7.5 Evaluation 181
7.5.1 Scenario 182
7.5.2 SLA 183
7.5.3 Smart contract generation 184
7.5.4 Chaincode deployment 185
7.5.5 IoT emulation 185
7.5.6 Results 187
7.6 Discussion 187
7.6.1 Limitation 189
7.7 Conclusion and future work 190
References 191

8 Intrusion detection and prevention in software-defined networks 195
Thomas Girdler, Celyn Birkinshaw, Wei Jie and Vassilios Vassilakis
8.1 Introduction 195
8.2 Basics of SDN 196
8.2.1 Conventional networking vs. SDN 196
8.2.2 SDN components 197
8.2.3 SDN controller 198
8.2.4 OpenFlow protocol 199
8.3 SDN-based cloud 199
8.4 SDN security 201
8.4.1 Application plane 201
8.4.2 Control plane 202
8.4.3 Data plane 202
8.5 Detecting ARP spoofing attacks in SDN 202
8.5.1 Overview of ARP spoofing detection technique 203
8.5.2 IDPS 203
8.5.3 Testbed 205
8.5.4 Intrusion detection – ARP spoofing 206
8.5.5 Intrusion detection – blacklisted MAC address 206
8.5.6 Intrusion prevention – ARP spoofing 208
8.5.7 Intrusion prevention – standard and spoofed ARP requests 210
8.6 Detecting port-scanning and DoS attacks in SDN 211
8.6.1 Design 211

8.6.2 Preparatory experiments 212
8.6.3 Attack experiments and results 217
8.7 Conclusion 218
Acknowledgements 219
References 219

9 Privacy challenges for Internet of Medical Things 223
Sabeen Tahir, Yinhao Li, Masoud Barati, Omer Rana, Rajiv Ranjan, Gagangeet Singh Aujla and Kwabena Adu Duodu
9.1 Introduction 223
9.1.1 General data protection regulation (GDPR) 225
9.2 Related work 226
9.3 GDPR compliance verification in IoMT 228
9.4 Blockchain-based IoMT system 230
9.4.1 Application requirements 230
9.4.2 System architecture 231
9.4.3 System execution workflow 233
9.4.4 Multi-layered architecture 235
9.5 Conclusion 236
Acknowledgments 236
References 237

10 Quality-aware Internet of Things applications 239
Kaneez Fizza, Abhik Banerjee and Prem Prakash Jayaraman
10.1 Human-in-the-loop vs. autonomic IoT applications – exemplary use case in smart parking 240
10.2 Quality of experience – background 241
10.3 Quality in IoT applications – a view of existing literature 242
10.3.1 Quality in human-in-the-loop IoT applications 244
10.3.2 Quality in autonomic IoT applications 245
10.4 Establishing the key factors impacting quality in IoT applications 245
10.4.1 Quality of data 247
10.4.2 Quality of Service 248
10.4.3 Quality of Processing 248
10.4.4 Quality of Device 248
10.4.5 Quality of Context 248
10.4.6 Quality of User Interface 249
10.4.7 Quality of Information 249
10.5 IoT platforms for developing quality-aware IoT applications 249
10.5.1 Conceptual framework to develop quality-aware IoT applications 250
10.6 Conclusion 251
References 252

11 IoT deployment and management in the smart grid **255**
Dilara Acarali, Sunny Chugh, K. Rajesh Rao and Muttukrishnan Rajarajan
11.1 Smart grid infrastructure 255
11.1.1 Key concepts 256
11.1.2 Grid architecture 256
11.1.3 Network architecture 258
11.2 Smart grid management 261
11.2.1 Management systems 261
11.2.2 Grid management using the cloud 263
11.2.3 Grid management using edge computing 264
11.3 Smart grid cybersecurity 265
11.3.1 Attacks in the field 266
11.3.2 Attacks on management 267
11.3.3 Defence mechanisms 268
11.4 Current challenges and future research 269
11.5 Conclusions 270
References 271

12 Consensus algorithm for energy applications: case study on P2P energy trading scenario **277**
Vidya Krishnan Mololoth, Saguna Saguna and Christer Åhlund
12.1 Introduction 277
12.2 Consensus mechanisms 278
12.2.1 Analysis of algorithms 279
12.3 An energy trading scenario 280
12.3.1 Using PoW consensus algorithm 282
12.3.2 Using PoA consensus algorithm 285
12.4 Discussion 285
12.5 Conclusion 286
References 286

13 Streamed gaming **289**
Dale Whinham, Yiang Lu, Zhaoran Wang, Jiaming Zhang, Richard Davison, Gary Ushaw and Graham Morgan
13.1 Introduction 289
13.2 Streaming video games 291
13.2.1 Origins 291
13.2.2 Streamed media 292
13.2.3 Gaming requirements 295
13.2.4 Platform expectations 296
13.2.5 Handling legacy titles 297
13.2.6 Genre and timeliness 298
13.2.7 Experience 301
13.2.8 Summary 301

13.3 Multiplayer streamed gaming 302
13.3.1 Distributed virtual environments 302
13.3.2 Interest management 303
13.3.3 Cloud-based game clients 303
13.3.4 Distributed physics 304
13.4 Conclusions 306
References 307

Index **313**

Preface

The Internet of Things which has emerged as a buzz word in the last decade is now transitioning into a main stream technology that current and in the future will underpin every aspect of our lives, societies, industries, and environment. In 2022, broadband IoT (4G/5G) reached 1.3 billion connections with wide-area use cases that require higher throughput, lower latency, and larger data volumes. As per recent reports from McKinsey, by 2025, IoT applications could have $11 trillion impact. Research driven by ambitious use cases and benefiting from innovation areas in components, systems, networking, and web technologies are further driving the development of new and innovation IoT applications.

As IoT applications continue to grow, and emergence of technologies such as edge, 5G/6G, the traditional model of cloud-based processing data processing is no longer scalable and hence needs to shift. Shifting data processing to the edge of the networks can help organisations take advantage of the growing number of IoT devices deployment, improve network speeds, reduce cost of processing, guaranteeing high degree of security, and enhance customer experiences. The scalable nature of edge computing makes it an ideal solution for fast-growing, agile organisation, especially if they are co-located edge and cloud data centres infrastructures.

The future design of the IoT applications will depend crucially on the development of sophisticated edge-cloud platforms and data centres architectures and infrastructure for smart objects, embedded intelligence, and smart networks. Most of the today's IoT application deployment are mainly focused on data collection from sensors, whereas in the future, real-time actuation and smart behaviour will be the key points. Further, research driven by ambitious use cases and benefits from innovations in key areas of systems, networking, and web will change the dimension of future IoT applications and systems in terms of scalability, heterogeneity, complexity, and dynamicity.

The main issue arising in using several computing models to support IoT applications is the management of different physical/virtual infrastructures (e.g., datacentres, edge devices, IoT devices, and gateways) according to specific application/service requirements (e.g., latency, data volume, responsivity, processing delay, etc.). In particular, it is often hard to determine a priori how to deploy the IoT applications into different infrastructures – since resources' availability, system load, and connectivity features can unpredictably change over the time.

Written and edited by an international team of experts in the field, every chapter submitted to the book has been critically evaluated by at least two expert reviewers. The critical suggestions by the reviewers certainly helped and influenced the authors

Chapter 4

Introduction to benchmarking IoT middleware platforms

Shalmoly Mondal[1], Prem Prakash Jayaraman[1], Alireza Hassani[2] and Pari Delir Haghighi[3]

4.1 Introduction

Internet of Things (IoT) refers to connecting various devices (commonly called IoT devices) over the Internet, which can interact with each other by sending and receiving data. These connected devices in the IoT ecosystem are expected to increase to 75.44 billion by 2025 [1]. This would be a threefold increase from the IoT installed base in 2019 [2]. Collecting these connected devices, a part of the IoT ecosystem, generates enormous data. IoT devices have the capability to monitor the environment, e.g., sensing the temperature of a building (sensing). This data is used by IoT applications to help in making a decision, such as turning on the heater when the temperature drops below a certain threshold (actuation). Hence, IoT aims to potentially resolve issues with minimum or no human intervention. More complex IoT applications use data analytics such as machine learning (ML) algorithms, natural language processing (NLP), and other complex analyses to process the IoT data to produce useful and actionable information that aids decision-making.

The plethora of IoT devices has resulted in an explosion of IoT application development. The significant advances in IoT technology have led to IoT applications being widely used in various scenarios ranging from the smart city [3], smart farming [4][5], to Industrial IoT (IIoT) solutions [6]. Industrial and transportation applications are currently dominating the IoT application areas [7]. For example, the IIoT application [6] aims to improve the productivity of factory workers in a meat processing plant by analyzing data and providing worker-specific predictions. This is achieved by analyzing accelerometer data generated by wearable watch-like IoT devices worn on each wrist by these factory workers. An ML-based activity recognition algorithm is being used to detect the various activities of the meat processing workers, like cutting or

[1]School of Science, Computing and Engineering Technologies, Swinburne University of Technology, Melbourne, Australia
[2]School of Information Technology, Deakin University, Australia
[3]Department of Human Centred Computing, Monash Data Futures Institute, Monash University, Melbourne, Australia

slicing chunks of meat. These activities are then translated to Key Performance Indicators (KPIs) for assessing the performance of individual workers. Another example can be taken of a smart parking application scenario [8], which provides a prediction mechanism for available parking spots by considering factors like the location, weather, and physical attributes of the car.

Smart farming is another area that takes advantage of IoT. There are many added benefits like being able to check soil fertility, analyze crop performance and provide recommendations. Various sensors are used for monitoring the soil fertility, to check the salinity, PH levels, and moisture levels in the soil, which can affect crop production. The data from these sensors can be analyzed to decide if irrigation is necessary and when would be the right time for cultivation. An IoT platform, SmartFarmNet [5] has been developed for smart farming applications which perform crop performance analysis and provide crop recommendations. Another scenario in the farming industry that leverages the benefits of IoT is monitoring the fertility of dairy cattle, which is an important aspect of dairy production. However, the decline in fertility of dairy cows has been a major concern for the dairy industry for a long time. IoT applications can be used for fertility management of dairy cattle [9]. Sensors are attached to the dairy cows, which periodically measure the heat status and notify farmers. IoT applications also find their use in health scenarios. For example, an IoT application can use a smartwatch sensor to predict blood alcohol content (BAC) to avoid drunk driving [10]. The application would be highly beneficial as it can warn its users if they become too intoxicated. Similarly, health vitals from a wearable IoT device combined with an activity recognition algorithm (based on ML) can be used to detect falls in elderly person [11].

As seen from the above examples of the different IoT applications, IoT provides enhanced services to applications connected to the internet using the data generated from the IoT devices. This IoT data can be heterogeneous, e.g., image or time-series sensor data. IoT is also characterized by diversity in various devices, communication, and network protocols. To cope with this diversity, IoT middleware platforms are increasingly being used to manage the devices, and data generated by them, and host IoT applications to facilitate the application development process [12]. The current research trend shows that the market for IoT middleware is expected to reach a value of USD 22.36 billion by 2025 [13]. IoT middleware provides several benefits, including the ability to break silos, manage the integration of data from different application domains, and enable IoT applications to share data between applications. Hence, with such rapid development of IoT applications, IoT middleware platforms are increasingly being used to expedite the application development and deployment process.

The performance of an IoT application depends on the choice of middleware platforms. Traditionally, users deploy IoT applications without sufficient knowledge about the IoT platform they should consider. With so many choices of existing IoT platforms, it is not clear how IoT application developers can make a choice of a platform that suits their needs. Hence, there is a need for a methodology to assess the performance of various IoT middleware platforms and to be able to compare the performance. The aim of this chapter is to give a brief overview of IoT middleware

platforms and their significance. We will also discuss the state-of-the-art benchmarking IoT middleware platforms and present the research gaps and challenges. The rest of the chapter is organized as follows. Section 4.2 describes a motivating scenario that highlights the significance of this research. Section 4.3 discusses the evolution of a middleware-based architecture of the IoT ecosystem. Section 4.4 provides an overview of the IoT middleware platforms and presents some commonly used open-source and commercial IoT platforms. Section 4.5 introduces benchmarking and the related work in benchmarking IoT middleware platforms. Section 4.6 discusses the challenges and open issues in the area. Section 4.7 concludes the chapter with a road map for future work.

4.2 Motivating scenario

Let us consider a traffic monitoring and road safety scenario, which monitors the traffic status and sends alerts to the drivers in case of road accidents or traffic congestion.

Figure 4.1 shows the different tasks involved in this application scenario. The IoT devices used in this scenario are: (i) surveillance cameras for capturing video data streams; (ii) traffic sensors (like ultrasonic sensors, radars, and loop detectors) monitor the data and get the number of vehicles; (iii) on-board sensors, speed sensors, and GPS-enabled vehicles calculate the speed and get the location of the vehicles. Surveillance cameras are mostly used to detect road traffic. The data from these devices is collected for processing. Pre-processing of the data, like aggregations, is done on the data to find the cumulative density (no. of vehicles) of a road and

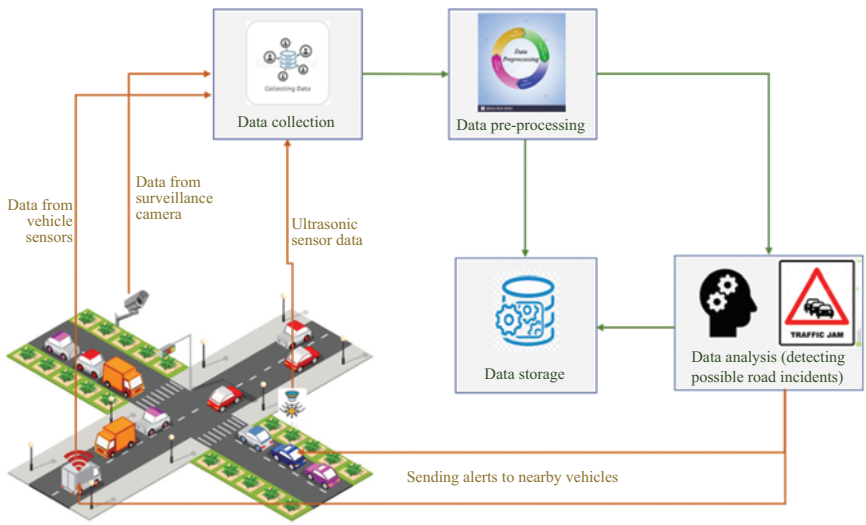


Figure 4.1 Traffic monitoring and road safety in smart cities

Table 4.1 Components and tasks involved in a traffic monitoring scenario

Num	Components	Description
1.	IoT devices	Capturing data from the environment including cameras, ultrasonic sensors, traffic light sensors, etc.
2.	Data collection	The data captured from the IoT devices are collected for further processing
3.	Data pre-processing	(i) Performing aggregation to find the cumulative density of a road; (ii) calculating the avg speed of the vehicles
4.	Data storage	Data from the sensors are stored to support further data analysis
5.	Data analysis	Transforming raw data captured from the sensors to detect possible road incidents and compute peak hours of traffic during different times of the day

calculate the avg speed of the vehicles. Data can either be analyzed in real-time to detect traffic events or stored at a repository for further analysis to detect traffic congestion or predict near misses. For example, traffic lights can be controlled if the number of vehicles exceeds the threshold. For example, the duration of the green signal can be increased. The traffic status (peak traffic hours/statistics of the traffic on the road) can be presented graphically to the users. All these computations are taken care of by IoT middleware. Hence, IoT middleware platforms play a key role in deploying IoT applications. They act as an intermediary and provide data to the IoT applications. The data from these various IoT devices can be sent to IoT middleware platforms using various communication protocols like MQTT, CoAP, AMQP, etc. After the data is received by the middleware, the data can be pre-processed, stored, or analyzed as seen in Figure 4.1. Table 4.1 briefly summarizes the functionality of each component involved in the above scenario. There are different components of a middleware platform that are responsible for all these different computations. We will discuss the different middleware components and their functionalities in detail in Section 4.3.

Considering the above example of the traffic monitoring scenario, where heterogeneous IoT data (e.g., IoT data from surveillance cameras, traffic sensors, onboard vehicle sensors) needs to be analyzed (using ML or complex event processing (CEP)) to predict events (e.g., road congestion or accidents) and send alerts to the users immediately. An IoT middleware platform can provide support to integrate the surveillance cameras, collect the data from them, store the data, provide support to run analysis to process image data, and provide the analyzed data to the IoT application. IoT middleware platforms such as Amazon Web Service [14], Microsoft Azure [15], FIWARE [16], Cumulocity [17], OpenIoT [18], to name a few offer various managed services which take care of the above-mentioned tasks. For example, AWS leverages managed AWS IoT services like IoT Core, IoT Greengrass, IoT Analytics, to collect, store, and analyze data. Similarly, Microsoft Azure provides services like IoT

Edge, IoT Event Hub, Azure Time-series insights for the same tasks. These diverse services offered by various middleware platforms make it a challenging task for IoT application developers to choose the IoT platform that suits the needs of their IoT application. Hence, despite the popularity and widespread use of IoT middleware platforms to host IoT applications, there is no systematic approach to compare the different IoT middleware platforms which can provide a common ground to enable an apple-to-apple comparison.

4.3 Middleware-based architecture of the IoT ecosystem

There exist several definitions of IoT middleware in the literature. The terms IoT middleware, IoT middleware platform, and IoT platform are generally used interchangeably. Razzaque *et al.* [19] have defined the IoT platform as "a software package that integrates devices, networks, and applications." In [20], middleware has been defined as "the software layer between technological and application layers." Bandyopadhyay *et al.* [12] defined IoT middleware as "a software platform providing an abstraction to applications from the things and offering multiple services."

In this section, we discuss the evolution of the middleware-based architecture of the IoT ecosystem and provide an overview of the architecture model considered in this study. The IoT ecosystem is complex, consisting of heterogeneous sources of data, and multiple layers of processing, and follows a layered architecture [21]. During the last decade, various architectures have been proposed by the research community. Figure 1.2 represents some of the most popular IoT architectures, namely (a) Three-layer [22], (b) middleware based [23], (c) SOA based [24], and (d) Five-layer [21] [25].

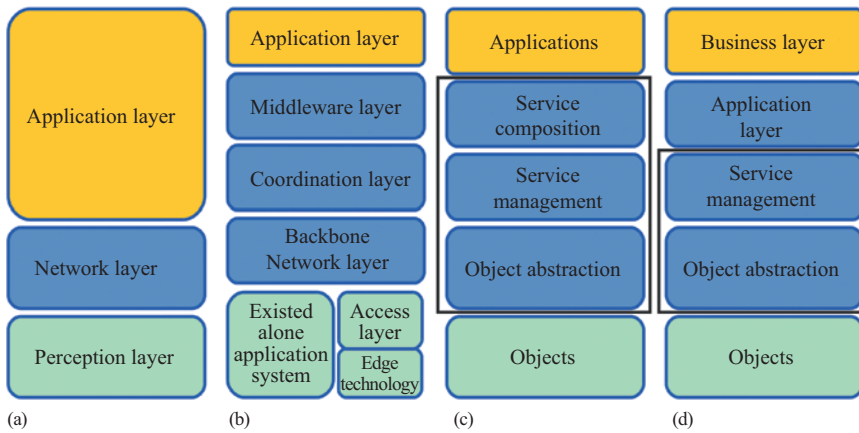


Figure 4.2 IoT architecture. (a) The three-layer IoT architecture. (b) Middleware based. (c) SOA based. (d) Five-layer [24].

The most basic architecture for the IoT ecosystem is a three-layer architecture consisting of a perception layer, a network platform layer, and an application layer (Figure 4.2(a)) [26]. The perception layer [22] is for identifying objects and gathering information. It consists of physical objects and sensor devices. The sensors can be RFID tags, 2D-barcode, cameras, GPS, or Infrared sensor subject to the objects identification method. The collected information from these devices is then passed to the network layer for its secure transmission to the information processing system. The network layer can also be called "Transmission Layer." Its main function is transmitting, and processing information obtained from the perception layer. The transmission medium can be wired, or wireless, and technology can be 3G, UMTS, Wi-Fi, Bluetooth, infrared, ZigBee, etc., depending upon the sensor devices [23]. The application layer includes applications that use IoT technology. The applications of IoT can be smart homes, smart cities, smart health, animal tracking, etc. [27].

The three layered architecture model has been improved in [21,23] with four and five-layered architectures. The middleware layer in Figure 4.2(b) provides functionality like data storage and processing. The IoT devices connect to the middleware and communicate with other devices through different communication protocols like MQTT, HTTP, and ACMP. The middleware receives the information from the IoT devices and the network layer and stores the collected data in the database. It performs data processing and computation on the data [23]. The object abstraction layer in Figure 4.2(c) transfers data produced by the objects layer to the service management layer through secure channels. Data can be transferred through various technologies such as RFID, 3G, GSM, UMTS, Wi-Fi, Bluetooth Low Energy, infrared, and ZigBee [24]. The object abstraction, service management, and service composition layer together in Figure 4.2(c) provide all capabilities that a middleware layer typically provides. Similarly, in Figure 4.2(d), the object abstraction, and service management layer can be called a middleware layer since these layers function as a middleware layer. It is shown by a black box in the figure. This layer enables the IoT application programmers to work with heterogeneous objects without consideration of a specific hardware platform.

We can see that there is no uniformity in the architectures described. Furthermore, the problem with most of the proposed IoT architecture is they are domain-specific (vertical IoT silos) [28,29]. Moreover, due to the variety of available IoT architectures, there arise interoperability issues [25], as no standard architecture has been officially adopted. Thus, a mutual understanding is needed to break the IoT Silos so that data can be reused, facilitate interoperability issues, and provide horizontal integration [24]. An architecture that can be used across several application domains is the key to ensuring mutual understanding. Realizing the importance of having a standard architecture for IoT, several IoT-related projects, co-funded by the European Commission under the Seventh Framework (FP7) Programme, were initiated to design a common architecture based on the needs of researchers and the industry. The SENSEI project [30], European Telecommunications Standards Institute (ETSI) [31], IoT-A [32] have been partially successful in defining architectures for different IoT applications. The authors of [33] presents an overview of the work done toward defining such a common architecture. ETSI Machine to machine (M2M) technical committee aimed to define the end-to-end

high-level system architecture for M2M. The Internet-of-Things Architecture (IoT-A) project aims to provide a general Architecture Reference Model (ARM) that can be used to derive concrete IoT architectures. While all the efforts are working towards a blueprint architecture, given the evolving nature of IoT, a standard architecture for the IoT ecosystem still exists that has been agreed on universally.

Based on the literature, most IoT applications use a middleware-based architecture. Considering the popularity of middleware and the ease of using them for hosting IoT applications, for this research, we consider a simple three-layer architecture model of the IoT system as recommended by IoT-A [32]. Figure 4.2 shows the layers of the architecture model used by most IoT applications consisting of a device layer, an IoT middleware platform layer, and the IoT application layer. The device layer is responsible for sensing and actuating. It consists of IoT devices that have integrated sensors and actuators. These IoT devices have the capability to register with the cloud securely and have connectivity options for sending and receiving data to and from the cloud. Depending on the type of IoT device, the information can be about location, temperature, orientation, motion, vibration, acceleration, etc. For example, in our traffic monitoring and road safety motivating scenario, the device layer consists of surveillance cameras for capturing video data streams, traffic sensors (like ultrasonic sensors, radars, and loop detectors) to get the number of vehicles crossing a particular road at any point of time, and onboard vehicle speed sensors and GPS to calculate the speed and location of the vehicles. The collected information is then passed to the middleware platform layer for further analysis. The IoT middleware platform layer is an intermediary between IoT devices and IoT applications. It provides functionalities to ease the development and deployment of IoT applications. The middleware platform layer consists of various components to perform different tasks. For example, it provides the ability to collect data from IoT devices and serve it to IoT applications. It can also process and analyze the data collected from IoT devices. The topmost layer, the application layer, is responsible for interacting with the end users, such as providing a response (analyzed data) to the user's query or issuing queries or actuation signals. It is in this layer that users deploy applications. We will analyze existing IoT platforms from the perspective of a middleware-based architecture and discuss the fundamental components of an IoT middleware platform in the next section.

4.3.1 Component and functionalities of IoT middleware platforms

In the previous section, we discussed the different layers in a middleware-based architecture of the IoT ecosystem. In this section, we focus on the fundamental components of the IoT middleware platform layer illustrated in Figure 4.3. The components were identified after analyzing various IoT middleware platforms. Some commonly used IoT middleware platforms have been discussed in Section 4.4.

Device integration and data ingestion: Device integration provides an interface for devices to communicate with the IoT middleware platform. Device integration often involves device registration, device discovery, and device authentication. Device registration makes tracking and monitoring the devices easy, as several devices exist for every IoT deployment. Registering each device to the IoT platform gives a unique

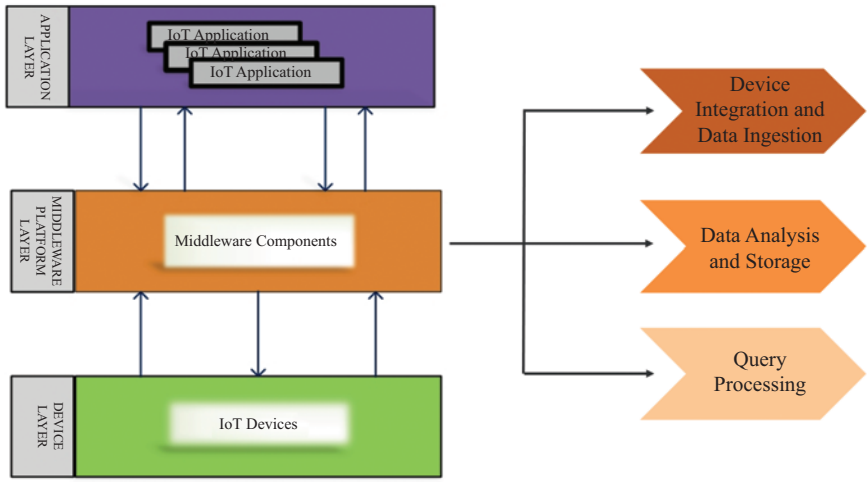


Figure 4.3 Fundamental components of the IoT middleware platform layer

identifier for secure communication with other devices and components of the IoT ecosystem. It also helps with authentication on a network. Device discovery allows users to know the properties and capabilities of IoT devices. Device authentication is required to verify the identity of the devices. After the devices have been registered into the platform, IoT data can be ingested from the devices to the middleware. This can be done using communication protocols like MQTT, CoAP, or AMQP. After the data is ingested, middlewares can transform the data for storage into data stores. One of the most common mechanisms used for data ingestion is Apache Kafka.

Data processing and storage: IoT data can be stored persistently for further analysis, or streaming data can be analyzed in real-time depending on the IoT applications. Applications like the traffic monitoring scenario discussed in Section 4.1 require real-time data analysis to obtain timely insights. This component provides both of these functionalities. Data storage acts as persistent storage for data from IoT devices for further analysis and querying. It stores the processed IoT data streams and historical data that have been cleaned to fit a relational schema. Data can either be stored in relational databases if ACID properties are a priority to speed and scalability, or they can be stored in No-SQL databases like MongoDB, Cassandra, and Hadoop, among others. Amazon S3 and Amazon Relational Database Service (RDS) are examples of data storage mechanisms provided by AWS. Similarly, FIWARE [16] uses a context management generic enabler to store, access, and analyze data. Common mechanisms used for data analysis are ML techniques like classification, clustering, and regression. Data analytics leads naturally to predictive analytics using collected data to predict what might happen. For example, a predictive analysis mechanism like image recognition can be used in the traffic monitoring scenario to predict the possibilities of road congestion. Similarly, in a mobile application for air quality monitoring [34] in a smart city environment, the mobile crowd-sensed data provided in

real-time by wearable sensors are used to analyze and infer air quality. The k-means clustering algorithm has been used to understand the correlation between the sensed variables like temperature, humidity, CO, and SO_2 and analyze their contribution to the air quality conditions.

Data retrieval and analysis: Query processing is a crucial factor in data processing for IoT applications, as most IoT applications are data-intensive, and they rely on querying as an elemental mechanism to access and retrieve data. Different types of queries can be issued depending on the IoT application running on the platform. There could be simple or complex queries, or push or pull-based queries. For example, an event-based IoT application will require a push-based query to get a suitable response. A simple push-based query would trigger a rule when any event occurs. For example, a query in such a situation would be "send an alert when there is road congestion, i.e., no_of_cars is greater than 20 on road_x."

4.4 IoT middleware platforms – a summary

With IoT platforms being widely used, various commercial and open-source middleware platforms are being used for hosting IoT applications.

4.4.1 Open source IoT middleware platforms

OpenIoT [18] is an open-source IoT platform that provides an integrated environment for deploying and managing IoT applications. It is licensed under Apache license 2.0. The fundamental concept here is virtual sensors. OpenIoT includes a sensor middleware that eases data acquisition from virtually any sensor. Various components serve different functionalities. OpenIoT uses X-GSN middleware to register the sensors, data acquisition, and sensor deployment. The sensor middleware contains a publish-subscribe middleware responsible for discovering and collecting data from mobile sensors (e.g., wearable sensors, built-in mobile devices). Data storage is enabled by a cloud database, which enables storage of data streams from the sensor middleware. The configuration and monitoring component enables visual management and configuration of functionalities over sensors and services deployed within the OpenIoT platform.

FIWARE [16] is another open-source IoT platform. FIWARE architecture has a set of components called General Enablers (GE). Each GE represents a FIWARE service. This rich suite of generic enablers deals with context management, interfacing with IoT systems, and data processing, analysis, and visualization. The *Orion Context Broker* Generic Enabler is the core and mandatory component of any FIWARE solution. It is a data/context management component. It enables us to manage context information in a highly decentralized and large-scale manner. The data storage mechanism used by this platform is a Context broker service. It uses a connector called *Cygnus* which is responsible for persisting or retrieving from a specific storage. The STH *Comet* Generic Enabler can store a short-term history of context data (typically months) on MongoDB. It can be used to manage the time-series data, e.g., reading

the temperature history of a sensor. Orion uses a Publish/Subscribe mechanism to store and retrieve context information. Orion can create context elements, update, and query them. The context elements are represented as JSON key-value pairs following the NGSIV2 standard. FIWARE NGSI v2 API is a Restful API enabling to perform updates, queries, or subscribe to changes on context information. FIWARE also provides several GE to interface with IoT and other systems. The *IDAS* Generic Enabler can connect IoT devices to the platform. IoT sensor enablement is responsible for integrating the sensors into the platform and registering sensor devices. Proton is used to register subscriptions with the context broker and to emit alerts once some unusual pattern in sensor data is detected.

Kaa [35] is an open-source server-based platform developed by CyberVision Corporation. It is suitable for enterprise-grade IoT applications and home-grown IoT projects and experiments. Kaa implements a microservice-based architecture. All Kaa microservices are available as docker images. Most microservices are written in Java, Go, and TypeScript, but users can also implement integrated microservices in Python and Scala. To use multiple services as one integrated platform, they can be deployed in a cluster. In simple terms, Kaa-based IoT solutions would be deployed as a cluster of microservices. One of the key concepts in Kaa is an *endpoint* which represents the things or devices of the IoT ecosystem. Another key concept is a *client*, essentially a gateway. Kaa client can be any software application that recognizes an endpoint and sends and receives data using IoT communication protocols. The communication protocols supported by this platform are HTTP, CoAP, MQTT, and WebSockets. *Endpoint Tokens* are used for identifying any device communicating with the platform. Kaa leverages multiple third-party services to carry out the platform's functionalities. For example, data storage is implemented by Cassandra, Influx, and MongoDB. Similarly, data analysis in Kaa is supported by integrating with Open Distro for ElasticSearch, a third-party open-source data analytics platform. The platform offers data visualization capabilities via the platform component *Web Dashboard Service*. Kaa also provides a pre-configured virtual environment called *Kaa sandbox* for small-scale development and testing. The sandbox can run either in a VirtualBox environment or on AWS EC2. Kaa also integrates business tools like SAP, Salesforce, etc.

ThingSpeak [36] is an open-source cloud-based platform that can be used for applications like smart farming and environment monitoring. Device integration to this platform can be done with MQTT and HTTP. Data processing can be implemented in Ruby, Python, and Node.js. It also provides APIs that support advanced data analysis using MATLAB® scripts. Additionally, it has a web interface that allows users to visualize the data using charts. Widgets can be created in JavaScript/HTML/CSS for a customized dashboard experience.

ThingsBoard [37] supports both cloud and on-premises deployments. ThingsBoard follows a microservice architecture. It supports REST communications with the server. Protocols like MQTT, CoAP, and HTTP can do device integration. The devices can be authenticated using access tokens. The platform leverages Redis, an in-memory data structure for data storage. Furthermore, ThingsBoard uses Zookeeper for coordination on the clusters. For data analysis, the platform uses a Rule Engine, a highly customizable CEP framework for building event-based workflows. ThingsBoard rule

engine consists of three main components: (i) *Message*, which is any incoming event. (ii) *Rule Node*, a function executed on an incoming message. (iii) *Rule Chain* are the nodes connected to send outbound messages to the next connected node. ThingsBoard integrates third-party services to operate the platform. For example, ThingsBoard uses Apache Kafka to persist incoming telemetry from HTTP/MQTT/CoAP transports until the rule engine processes it. ThingsBoard also uses Kafka for some API calls between micro-services. The platform also allows us to create customizable IoT dashboards for data visualization.

Node-RED [38] is a Node.js-based middleware platform from IBM. It implements an actor-based architecture. The basic concept of this platform is a *node*, which encapsulates a block of JavaScript codes that performs a specific function. Each node can be called an actor. Node-RED is a lightweight platform built on Node.js and features an event-driven, non-blocking model making it ideal for running on edge devices like Raspberry Pi, Beaglebone Black, and Arduino. The platform supports communication protocols like MQTT and Websockets. The key feature of Node-RED is a visual tool that allows flow-based programming for wiring hardware devices. The tool allows drag-and-drop functionalities to wire up any IoT application. Some other concepts associated with tools are: (i) *Flow*, which is a set of connected nodes. (ii) *Wires* connect the nodes and represent how messages pass through the flow. (iii) *Context*, which is a way to store information between the nodes. (iv) *Message*, which passes between the nodes in a flow.

Agri-IoT [39] is another open-source platform for smart farming applications. The platform has components like a device manager for device authentication and identity, a discovery module for registration and discovery of IoT devices, data aggregation to deal with large volumes of data using time series analysis, and data compression techniques to reduce the size of raw sensory observations, data federation which deals with users' requests. It translates users' requests into RDF Stream Processing (RSP) queries and evaluates them to obtain results. Event detection provides tools for processing annotated and aggregated data streams to obtain farm events, such as the need for irrigation, sick animals, or pest identification in crops. Real-time adaptive reasoning considers the farmer's preferences and dynamic contextual farm-related information (represented by real-time events). To provide optimal decision support in real-time, a dashboard provides immediate and intuitive visual access to the results of processing and analysis of data and events.

Context-as-a-service (CooaS) [29] is a context management platform that manages the interaction and enables applications and services to share context seamlessly without requiring the manual integration of IoT silos. The middleware has various components, namely Security and Communication Manager, Context Query Engine (CQE), Context Storage Management System (CSMS), and Context Reasoning Engine (CRE).

4.4.2 Commercial IoT platforms

AWS IoT [14] is a commercial platform that allows communication between Internet-connected devices such as sensors, actuators, smart devices, and the AWS. The device

gateway enables communication between publishers and subscribers. It currently supports publishing and subscribing over secure MQTT, WebSockets, and HTTPS. Device software like *AWS IoT Greengrass* also provides local data collection and analysis at the edge, even without internet connectivity. *AWS IoT* Core is a managed service that allows connected devices like cars, lights, smart speakers, cameras, fans, and any other item to interact with cloud applications and other devices quickly and securely. Devices are registered via Device Registry. The Device Shadow feature enables cloud and mobile applications to query data sent from the devices and send commands to devices using a simple REST API. AWS IoT Core rules engine allows filtering, transforming, and acting upon device data on the fly. Amazon IoT uses a variety of durable storage solutions provided by Amazon, like *Amazon RDS*, *Amazon Dynamo Db*, etc. Amazon IoT platform is a highly scalable and can handle a high streaming data rate. AWS uses a managed service called *Amazon Kinesis Firehose* to ingest real-time streaming and bulk data. Amazon Kinesis supports streaming data, enabling us to get timely insights and react quickly to the latest information from IoT devices. Amazon Kinesis integrates directly with the AWS IoT rules engine, creating a seamless way of bridging from a lightweight device protocol of a device using MQTT with the internal IoT applications that use other protocols.

Azure IoT [15] is a commercial cloud-based IoT platform providing scalable Internet of Things (IoT) applications using the Azure IoT managed and platform services. Azure IoT simplifies creating, securing, and managing devices across the entire device lifecycle. *Azure IoT Hub* messaging provides device-to-cloud and cloud-to-device communications. Devices can communicate with the IoT Hub over HTTP, MQTT protocols. The stream processor component in Azure IoT processes large streams of data records and evaluates rules for those streams. Azure uses a data factory or *databricks* for ingesting data. For data storage, Azure IoT integrates warm and cold storage services provided by Microsoft Azure. Azure supports data analysis by the ML subsystem, enabling systems to learn from data and experiences and act without being explicitly programmed. The user management subsystem allows the specification of different capabilities for users and groups to perform actions on devices (e.g., command and control such as upgrading firmware for a device) and capabilities for users in applications.

Cumulocity IoT [17] is another platform in this category, which provides support for IoT device integration, device management, offers streaming and batch analytics, and data visualization capabilities. It has a built-in extensive domain model which stores the domain data. The model can be extended for further uses and is complementary to SSN. Cumulocity also provides API for extending the existing functionality or interfacing Cumulocity IoT with other IT services such as ERP or CRM systems. It is specifically designed to work with Mobile networks. The platform also allows interfacing of IoT devices through software agents. The platform currently supports two main types of agent architecture: *server-side agents* and *device-side agents*. Server-side agents are run in a cloud, hosted on Cumulocity IoT as microservices, whereas device-side agents run on a device in the sensor network. Cumulocity provides a high-level real-time processing language to run IoT business logic. *Apama's Event Processing Language (EPL)* can be used with Cumulocity for real-time processing.

The cockpit application feature provides data visualization functionality. It can be used to manage and monitor IoT data.

ThingWorx is a cloud-based M2M platform by PTC targeted for IIoT solutions. Device integration in this platform is supported by REST API. It provides a tool *AlwaysOn* for communication between edge devices and the platform. Authentication of devices for communicating with ThingWorx is supported by *Applicationkeys*, which are security tokens. Integrates data from various sources like edge devices, and data from sales and ERP systems. SDKs supported by the platform are C, Java, and .NET SDK's. *Edge Microserver (EMS)* allows edge devices and data stored to connect to the platform. The C SDK is the basis for EMS. *Persistence Provider* is a concept in ThingWorx that enables connecting to a data store and performing CRUD operations on the data. The data storage options offered by this platform are H2, PostgreSQL, Microsoft SQL Server, Azure SQL database, and InfluxDB. The persistence provider framework of the ThingWorx platform can be configured to use multiple data stores for a given data provider. This feature can be leveraged to distribute the data ingestion and query processing workload to multiple data stores to overcome the typical RDBMS vertical scalability limitations. It provided data analysis capabilities that allow building predictive models using AI and ML technologies. For data visualization, the platform provides a tool, *mashup widgets*, for creating the functional user interface. Have a dedicated application store called PTC marketplace.

SmartFarmNet [5] is another IoT platform developed for smart framing to increase farm productivity. Currently, crop performance is collected manually under various conditions like soil quality, etc., which is slow. SmartFarmNet has been developed to automate the collection of environmental, soil, fertilization, and irrigation data; automatically correlate such data and filter-out invalid data from the perspective of assessing crop performance; and compute crop forecasts and personalized crop recommendations for any farm.

4.5 Benchmarking IoT middleware platforms

In this section, we introduce benchmarking, terminologies related to benchmarking, and the state-of-the-art in benchmarking IoT middleware platforms.

4.5.1 Introduction to benchmarking

The term benchmarking is defined in the Oxford Dictionary as "Evaluating something by comparison with a standard." Xerox's [40] formal definition of benchmarking is "the continuous process of measuring products, services, and practices against the company's toughest competitors or those companies known as industry leaders." In short, it means "finding and implementing best practices." The practice of benchmarking is now being applied to various other domains. Camp [41] defines benchmarking as creating performance results for a given set of tests. These tests represent the performance of an entire application or a component of the application

(also known as Micro Benchmarks). The performance results are used as an indicator of how well the application or application components perform given a specific configuration.

Benchmarking vs. profiling

Benchmarking is often confused with the term profiling. Both benchmarking and profiling are used to measure how a system is performing in varying conditions [41]. However, there is a subtle difference between benchmarking and profiling. While benchmarking answers the question, "How well a system performs?" profiling tells us why a system performs how it performs. The benchmark results are used to compare different systems, whereas the result sets of profiling are used to determine the problematic component of a system. Moreover, in benchmarking, it is essential to define a standard set of datasets and queries to measure the performance of the different systems (e.g., IoT middleware platforms) under consideration fairly. In this research, we consider benchmarking as "*A mechanism by which we can establish benchmarks for different IoT middleware platforms running IoT applications by using a standard set of IoT data and queries*." The benchmarks must be repeatable and can provide the foundations for conducting empirical, experimental comparisons between IoT middleware platforms. By establishing benchmarks, we mean measuring the performance of IoT middleware platforms and comparing them based on certain metrics like query response time, data ingestion rate, security, and accuracy of the analytical mechanism being used by the IoT application.

Benchmarking metrics: For any benchmarking methodology, we need to define metrics or criteria based on which the performance of a system will be evaluated. We call these "benchmarking metrics." These metrics could be performance metrics like throughput, and response time, security metrics like data encryption, and device authentication, or resource usage like CPU, and memory utilization of the underlying resources.Other metrics include scalability, reliability, and availability. Some of the metrics are discussed below:

- *Response time*: average time difference between the time an input arrives and the time a response/outcome is generated.
- *Throughput*: the number of tasks completed per unit of time.
- *Execution time*: time to execute a given task.
- *Data ingestion rate*: time difference between when the data is being generated and when it is being inserted into the platform.
- *Supported query load*: the maximum input a system can process while meeting specific constraints like response time.
- *Scalability*: a system can handle many requests simultaneously.
- *Reliability*: [42] how a system can operate without failure for a given time.
- *Availability*: [42] time for which the system is accessible to the user.

4.5.2 State-of-the-art in benchmarks in IoT

In [43], two middleware platforms FIWARE and M2M have been compared based on a set of qualitative and quantitative metrics. The data used for conducting the benchmarks is a large publish-subscribe dataset from a traffic monitoring scenario that publishes the average traffic speed in each city street on an hourly basis. However, the middleware platforms have been considered a black box without considering the internal implementation. The authors reported that the middleware performance could depend on its different components like the data storage or the underlying communications protocol. However, it only focuses on publish/subscribe middleware platforms. In another work by Medvedev *et al.* [44], OpenIoT has been evaluated based on its data ingestion and storage capability.

Similarly, in another case study by Salhofer *et al.* [45], the FIWARE platform has been thoroughly analyzed and evaluated from an application deployment point of view. However, no performance and load test has been conducted. The web interface falls short on some fundamental functionalities. For example, it is currently not possible to modify, delete, or view existing permissions, which do not make the application fit for real-life use.

Agarwal *et al.* [46] provide a layer-wise comparative analysis of the IoT middleware and define some criteria for the selection of the right IoT middleware platform like availability, deployment type, and pricing model. Cruz *et al.* [47] give a performance evaluation study of a few existing middleware platforms. Qualitative and quantitative metrics have been proposed to evaluate the performance of the IoT Middleware platforms. The proposed metrics were tested with five middleware platforms: InatelPlat, Konker, Linksmart, Orion+STH, and Sitewhere. It has been reported that the different middleware platforms performed differently depending on the metrics. For example, in case of low throughput and packet size, Konker and Linksmart performed better, whereas when error percentage is the priority, Orion +SSH gives the best performance with less than a 1% error rate. Sitewhere operated well with concurrent users.

Some other research efforts towards IoT benchmarks include some benchmarking suites like IoTBench [52], a benchmark suite targeting the IoT edge devices. IoTBench includes seven representative applications from three categories that exhibit diversity in IoT. Shukla *et al.* [51] have proposed RIoTBench, a novel benchmarking suite for evaluating the data ingestion capabilities for IoT Applications. The benchmarking suite comprises of micro- and application-level benchmarks for the evaluation. The conventional processing and analytic tasks that are carried out over real-time IoT data streams have been categorized based on domain requirements. However, the benchmarking suite is limited to DSPS applications that are centrally hosted in the cloud and do not integrate edge or fog. Arlitt *et al.* [50] have introduced IoTABench, a benchmark suite for evaluating data analytics capability. The use case to demonstrate their proposed benchmark is of a smart metering scenario. For the dataset, they have used smart meter data which is generated using a Markov chain-based synthetic data generator. The benchmark consists of synthetic data, and six analytical queries: total readings, total consumption, peak consumption, top consumers, and time of

Table 4.2 Summary of the existing benchmark studies in IoT

Related works	Benchmarking targets	Benchmarking metrics	Capturing application requirements	IoT dataset	IoT queries
BenchIoT [48]	Device layer	Resource usage, performance metrics	NS[b]	NC[c]	NC
TPCx-IoT [49]	Device layer	IoTps[a], throughput	NS	Electric power stations	Real-time analytic queries
IoTAbench [50]	IoT middleware platform layer	NS	NC	Smart meter data	Statistical queries
RIoTBench [51]	IoT middleware platform layer	Latency, throughput, CPU, memory utilization	NS	NC	NC

Source: Text roughly sketched.
[a]Performance metrics.
[b]Not specified.
[c]Not considered.

usage billing. In this benchmark suite, each benchmark represents a distinct use case. However, currently, only one use case has been implemented.

TPC council, which has been successful in creating benchmarks for databases, introduced TPCx-IoT [53], a benchmark for comparing the performance of IoT gateways. They have implemented a use case of power substations of Electric utility providers. They have used a dataset of an electric power substation. The TPCx-IoT workload generator is based on the Yahoo! Cloud Serving Benchmark framework. The TPCx-IoT workloads consist of data ingestion and concurrent queries simulating workloads on typical IoT Gateway systems. The primary metrics used in TPCx-IoT benchmark in the IoTps (performance metric) and $/IoTps (price-performance metric).

After analyzing the existing IoT benchmarks, we have observed that existing research in benchmarking is fragmented, i.e., existing approaches have specific benchmarking targets. By benchmarking targets, we mean the layer of the IoT ecosystem being addressed. For example, in Table 4.2, BenchIoT [48] and TPCx-IoT [49] target the IoT device layer while IoTAbench [50] and RIoTbench [51] focuses on IoT middleware layer. Furthermore, other research in benchmarking for IoT middleware platforms focuses on selected components of middleware platforms. For example, IoTAbench [50] specifically data analysis, whereas RIoT bench is targeted towards data ingestion of the middleware layer. Additionally, some of the benchmarks are application-specific and cannot be applied to more than one use-case [50]. To address these gaps in the literature, we need a comprehensive approach that can be used to

benchmark IoT middleware platforms. Hence, a holistic framework that can generate meaningful benchmarks will allow us to compare these different IoT middleware platforms and enable the users to select the appropriate platform for their application deployment. While there has been a lot of work done in related areas like big data, stream processing, etc., there is limited work for IoT. To accelerate the research effort in that direction, we have identified three factors that we think should be considered while developing a methodology for benchmarking IoT middleware platforms. We will discuss them in the next section.

4.6 Towards a holistic framework for benchmarking IoT middleware – challenges and open issues

To develop a holistic framework for benchmarking IoT middleware, we discuss the identified factors and their significance below. We also discuss the challenges and the open issues.

Capturing IoT application requirements: IoT applications are being used in various domains, wherein the requirements of IoT applications vary across these domains. As an example, a real-time application such as the traffic monitoring motivating scenariohas requirements like near-real-time data processing and low latency. Whenever any incident occurs, alerts should be broadcasted to the nearby drivers near an accident location as soon as possible. When deployed in an IoT middleware platform, such applications need to deliver outcomes within certain IoT application requirements. For example, notify the user within "x" milliseconds of any road event occurring, alerts/warnings should be sent, and the nearby drivers should be notified. However, most existing benchmarks do not clearly define the application requirements. While there is some related work that makes the best effort in defining IoT application requirements, this cannot be directly adopted for benchmarking purposes.

Generating IoT dataset: The second criterion is access to an IoT dataset for creating benchmarks. IoT data is different from traditional data. While traditional data is static, IoT data is dynamic, real-time, and changing frequently. In traditional cases, like in the case of database benchmarks, data semantics is unimportant. In the case of IoT applications, we want to know the context of the application, and based on the context, we want to generate datasets that represent the IoT applications. Creating benchmarks for IoT middleware platforms for IoT use cases requires access to large volumes of data generated by IoT devices. However, real data may not always be available to represent events such as earthquakes or road accidents where the velocity of data is far greater. While benchmarks in databases have well-established standard datasets, there is limited literature on IoT benchmarks that focus on generating data for IoT applications. Table 4.2 shows that none of the existing benchmarks takes into consideration the IoT application requirement in being able to generate relevant IoT datasets and IoT queries.

Designing benchmark queries: Queries are another integral part of any benchmarking framework. Limited research efforts exist in designing queries for creating benchmarks in IoT. Groups like TPC that create industry-standard benchmarks have

developed such queries for database systems. Similarly, such business queries have been created for the BigBench [54] workload for Bigdata systems. However, we are unaware of such query sets in the IoT domain. The only benchmark by TPC in the IoT domain is TPCx-IoT [49], the first industry-standard benchmark for comparing the performance of IoT gateways. The TPCx-IoT workloads consist of concurrent queries simulating workloads on typical IoT Gateway systems. Other research [50] uses simple statistical queries that are insufficient to represent complex IoT application scenarios. Different IoT applications require queries with different complexities and richness. In a smart parking scenario where a vehicle needs to find a suitable parking spot, if we assume the IoT entity making a query is a car, the query might have attributes like the car's location, which vary. We need to simulate or generate the change of location in the query. Hence, it is evident that traditional database queries will not suffice in creating benchmarks for IoT. We need to generate more high-level queries like context-aware queries (queries with geospatial functions, windowing functions, etc.). Most research in IoT benchmark, do not address this diversity and complexity in query generation. The complexity or richness of queries can impact the performance of benchmarks. This is an open problem [8], and one of the challenges would be generating queries that consider the query richness and diversity.

4.7 Conclusion

Currently, there are several IoT middleware platforms that facilitate connecting to IoT devices, managing the data generated by these devices, and hosting IoT applications. These platforms serve a wide variety of IoT applications with varying requirements. However, there are no standard means to benchmark the IoT middleware platforms. Most research in this area has focused on benchmarking cloud-based services. However, in the current literature, there are limited studies on creating holistic benchmarks for IoT middleware platforms.To develop and deploy IoT applications, choosing from the plethora of existing IoT middleware platforms is a challenge. A holistic and reliable IoT benchmarking framework will allow us to compare these different IoT middleware platforms and enable the users to select the most suitable and optimal IoT middleware platform according to their applications' requirements. We have highlighted in Section 4.6 the gaps in existing research and the challenges. We have also identified some criteria that will address the existing gaps in the literature, including (a) capturing the IoT application requirements; (b) generating IoT data taking into consideration the application's context, e.g., generating event-based data that represents an activity walking; and (c) devise IoT queries that provide a rich characterization of IoT applications.

List of abbreviations

Abbreviations	**Meaning**
IoT	Internet of Things
ML	Machine learning

AMQP	Advanced message queuing protocol
API	Application programming interface
ARM	Architecture reference model
AWS	Amazon Web Service
CB	Context broker
CEP	Complex event processing
CoAP	Constrained application protocol
CRE	Context reasoning engine
DSPS	Distributed stream processing systems
EPL	Event processing language
ERP	Enterprise resource planning
GIS	Geographic information system
GE	Generic enabler
HTTP	Hypertext transfer protocol
M2M	Machine to machine
MQTT	Message queuing telemetry transport
REST	Representational state transfer
RDF	Resource description framework
RFID	Radio-frequency identification
RSP	RDF stream processing
SOA	Service oriented architecture
TPC	Transaction Processing Performance Council
UMTS	Universal mobile telecommunications system
YCSB	Yahoo Cloud Serving Benchmark

References

[1] Department SR. Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025; 2016 (accessed September 3, 2020). https://www.statista.com/statistics/471264/iot-number-of-connected-devices-worldwide.

[2] News IB. The IoT in 2030: 24 billion connected things generating 1.5 trillion dollar; 2020 (accessed November 1, 2020). https://iotbusinessnews.com/2020/05/20/03177-the-iot-in-2030-24-billion-connected-things-generating-1-5-trillion.

[3] Ta-Shma P, Akbar A, Gerson-Golan G, *et al.* An ingestion and analytics architecture for iot applied to smart city use cases. *IEEE Internet of Things Journal*. 2017;5(2):765–774.

[4] Ahmed N, De D, and Hussain IJIIoTJ. Internet of Things (IoT) for smart precision agriculture and farming in rural areas. *IEEE Internet of Things Journal*. 2018;5(6):4890–4899.

[5] Jayaraman PP, Yavari A, Georgakopoulos D, *et al.* Internet of things platform for smart farming: Experiences and lessons learnt. *Sensors*. 2016;16(11):1884.

[6] Forkan ARM, Montori F, Georgakopoulos D, *et al.* An industrial IoT solution for evaluating workers' performance via activity recognition. In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE; 2019. p. 1393–1403.
[7] Analytics I. Top 10 IoT applications in 2020; 2020 (accessed Jan 18, 2020). https://iot-analytics.com/top-10-iot-applications-in-2020/.
[8] Medvedev A, Hassani A, Zaslavsky A, *et al.* Benchmarking IoT context management platforms: High-level queries matter. In: *2019 Global IoT Summit (GIoTS)*. IEEE. p. 1–6.
[9] Kamilaris A, Gao F, Prenafeta-Boldu FX, *et al.* Agri-IoT: a semantic framework for Internet of Things-enabled smart farming applications. In: *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*. IEEE; 2016. p. 442–447.
[10] Ngu AH, Gutierrez M, Metsis V, *et al.* IoT middleware: a survey on issues and enabling technologies. *IEEE Internet of Things Journal*. 2016;4(1):1–20.
[11] Jayaraman PP, Forkan ARM, Morshed A, *et al.* Healthcare 4.0: a review of frontiers in digital health. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*. 2020;10(2):e1350.
[12] Bandyopadhyay S, Sengupta M, Maiti S, *et al.* Role of middleware for internet of things: A study. *International Journal of Computer Science and Engineering Survey*. 2011;2(3):94–105.
[13] Wire B. Global IoT Middleware Market (2020 to 2025) – Growth, Trends, and Forecasts; 2020 (accessed Jan 18, 2020). https://www.businesswire.com/news/home/20201229005254/en/Global-IoT-Middleware-Market-2020-to-2025—Growth-Trends-and-Forecasts—ResearchAndMarkets.com/.
[14] Amazon Web Service IoT; (accessed July 31, 2020). https://aws.amazon.com/iot.
[15] Microsoft Azure IoT; (accessed July 31, 2020). https://azure.microsoft.com/en-au/services/iot-hub.
[16] Fiware; (accessed July 31, 2020). https://www.fiware.org.
[17] Cumulocity IoT; (accessed October 15, 2020). https://www.softwareag.cloud/site/product/cumulocity-iot.html.
[18] Open IoT; (accessed October 15, 2020). http://www.openiot.eu.
[19] Razzaque MA, Milojevic-Jevric M, Palade A, *et al.* Middleware for Internet of Things: a survey. *IEEE Internet of Things Journal*. 2015;3(1):70–95.
[20] Fersi G. Middleware for Internet of Things: a study. In: *2015 International Conference on Distributed Computing in Sensor Systems*. IEEE; 2015. p. 230–235.
[21] Tan L and Wang N. Future Internet: The Internet of Things. In: *2010 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE)*, vol. 5. IEEE; 2010. p. V5-376–V5-380.
[22] Wu M, Lu TJ, Ling FY, *et al.* Research on the architecture of Internet of Things. In: *2010 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE)*, vol. 5. IEEE; 2010. p. V5–484.
[23] Khan R, Khan SU, Zaheer R, *et al.* Future Internet: the Internet of Things architecture, possible applications and key challenges. In: *2012 10th*

International Conference on Frontiers of Information Technology. IEEE; 2010. p. 257–260.

[24] Al-Fuqaha A, Guizani M, Mohammadi M, *et al.* Internet of Things: a survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*. 2015;17(4):2347–2376.

[25] Di Martino B, Rak M, Ficco M, *et al.* Internet of Things reference architectures, security and interoperability: a survey. *Internet of Things*. 2018;1:99–112.

[26] Yang Z, Yue Y, Yang Y, *et al.* Study and application on the architecture and key technologies for IOT. In: *2011 International Conference on Multimedia Technology*. IEEE; 2011. p. 747–751.

[27] Burhan M, Rehman RA, Khan B, *et al.* IoT elements, layered architectures and security issues: a comprehensive survey. *Sensors*. 2018;18(9):2796.

[28] Zaslavsky A and Jayaraman PPJU. Discovery in the Internet of Things: the Internet of Things (ubiquity symposium). *Ubiquity symposium*. 2015; 2015(October):1–10.

[29] Hassani A, Medvedev A, Haghighi PD, *et al.* Context-as-a-service platform: exchange and share context in an IoT ecosystem. In: *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE; 2018. p. 385–390.

[30] Presser M, Barnaghi PM, Eurich M, *et al.* The SENSEI project: integrating the physical world with the digital world of the network of the future. *IEEE Communications Magazine*. 2009;47(4):1–4.

[31] Grieco LA, Alaya MB, Monteil T, *et al.* Architecting information centric ETSI-M2M systems. In: *2014 IEEE International Conference on Pervasive Computing and Communication Workshops (PERCOM WORKSHOPS)*. IEEE; 2014. p. 211–214.

[32] De Loof J, SAP CM, Meissner S, *et al.* Internet of Things—architecture IoT-A deliverable D1. 5–Final Architectural Reference Model for the IoT v3. 0.

[33] Krčo S, Pokrić B, and Carrez F. Designing IoT architecture (s): a European perspective. In: *2014 IEEE World Forum on Internet of Things (WF-IoT). IEEE; 2014. p. 79–84*.

[34] Hromic H, Le Phuoc D, Serrano M, *et al.* Real time analysis of sensor data for the Internet of Things by means of clustering and event processing. In: *2015 IEEE International Conference on Communications (ICC)*. IEEE; 2015. p. 685–691.

[35] Kaa Platform; 2021 (accessed July 3rd, 2021). https://www.kaaiot.com/.

[36] ThingSpeak; 2021 (accessed July 3rd, 2021). https://thingspeak.com/pages/learn_more.

[37] Thingsboard; 2021 (accessed July 3rd, 2021). https://thingsboard.io/.

[38] Node-RED; 2021 (accessed July 3rd, 2021). https://nodered.org/docs/.

[39] Kamilaris A, Gao F, Prenafeta-Boldu FX, *et al.* Agri-IoT: a semantic framework for Internet of Things-enabled smart farming applications. In: *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*. IEEE; 2016. p. 442–447.

[40] Camp RC. A bible for benchmarking, by Xerox. *Financial Executive*. 1993; 9(4):23–28.

[41] Kruckenberg M and Pipes J. Benchmarking and profiling. *Pro MySQL*. 2005. p. 189–234.

[42] Garg SK, Versteeg S, and Buyya R. Smicloud: a framework for comparing and ranking cloud services. In: *2011 Fourth IEEE International Conference on Utility and Cloud Computing*. IEEE; 2011. p. 210–218.

[43] Aguiar A and Morla R. Lessons learned and challenges on benchmarking publish-subscribe IoT platforms. In: *Proceedings of the 2nd Workshop on Benchmarking Cyber-Physical Systems and Internet of Things*; 2019. p. 24–29.

[44] Medvedev A, Hassani A, Zaslavsky A, *et al.* Data ingestion and storage performance of iot platforms: Study of openiot. In: *International Workshop on Interoperability and Open-Source Solutions*. Springer. p. 141–157.

[45] Salhofer P. Evaluating the FIWARE Platform. In: *Proceedings of the 51st Hawaii International Conference on System Sciences*; 2018.

[46] Agarwal P and Alam M. Investigating IoT Middleware Platforms for Smart Application Development. In *Smart Cities—Opportunities and Challenges: Select Proceedings of ICSC 2019*, pp. 231–244. Springer Singapore, 2020.

[47] da Cruz MA, Rodrigues JJ, Sangaiah AK, *et al.* Performance evaluation of IoT middleware. *Journal of Network and Computer Applications*. 2018;109:53–65.

[48] Almakhdhub NS, Clements AA, Payer M, *et al.* Benchiot: a security benchmark for the Internet of Things. In: *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE; 2019. p. 234–246.

[49] TPCx-IoT); 2021 (accessed June 1, 2021). http://tpc.org/tpcx-iot/default5.asp.

[50] Arlitt M, Marwah M, Bellala G, *et al.* Iotabench: an Internet of Things analytics benchmark. In: *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*. p. 133–144.

[51] Shukla A, Chaturvedi S, Simmhan YJC, *et al.* Riotbench: an IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience*. 2017;29(21):e4257.

[52] Lee CI, Lin MY, Yang CL, *et al.* IoTBench: a benchmark suite for intelligent Internet of Things edge devices. In: *2019 IEEE International Conference on Image Processing (ICIP)*. IEEE; 2019. p. 170–174.

[53] Poess M, Nambiar R, Kulkarni K, *et al.* Analysis of tpcx-iot: the first industry standard benchmark for IoT gateway systems. In: *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE; 2018. p. 1519–1530.

[54] Wang L, Zhan J, Luo C, *et al.* Bigdatabench: a big data benchmark suite from internet services. In: *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE; 2014. p. 488–499.

Index

ACMP 80
actor 85
actuator 19
address resolution protocol (ARP) spoofing 195
advanced metering infrastructure (AMI) 257
advanced persistent threat (APT) 267
Agri-IoT 85
allocation algorithms 59
  heuristic algorithm 60
  joint optimization 63–4
  low-complexity algorithms 60–2
  optimized allocation 62–3
  simulation results 64–6
Amazon Kinesis Data Firehose 86, 111–12
Amazon S3 118
Amazon Web Service 78
AMQP 78
Apache HBase 118
Apache Kafka 82, 85
Apache Spark 114
Apache storm 114
Apama's Event Processing Language (EPL) 86
Apple 292
application plane 201–2
application programming interfaces (APIs) 198, 225
Architecture Reference Model (ARM) 81
Arduino 85
artificial intelligence 224
attribute-based encryption (ABE) 227
Aura Projection 305
autonomous database (AutoDB) 27
AWS IoT 85–6
Azure IoT 86
Azure SQL database 87

base band unit (BBU) 55
BaseScheduler class 45
batch processing services 114–16
Beaglebone Black 85
benchmarking 87
  metrics 88
  vs. profiling 88
  state-of-the-art in 89–91
big data (BD) software 110
Bitcoin 277
blockchain 169, 226, 277
  assets 138–9
  deployment to 142–3
  design and implementation 172
  energy trading using 281
  evaluation 181
    chaincode deployment 185
    IoT emulation 185–7
    scenario 182–3
    SLA 183–4
    smart contract generation 184
  Hyperledger Fabric 170–1
  IoT 167–8
  limitation 189–90
  SLAs 168–9, 173–4
    FromSLAToSmartContract library 174–81
    From SLA to smart contract Java library 174
  smart contracts 169–70
  structure 170

blockchain technology 127, 166
blood alcohol content (BAC) 76
Bluetooth 224, 260
Bluetooth low energy (BLE) 260
building EMS (BEMS) 262
business area network (BAN) 259
Business Process Model and Notation (BPMN) 134

Canopy algorithms 117
capital expenditures (CAPEX) 55
carbon footprint 2
cardiovascular disease 182
Cassandra 118
certificate authority (CA) 233
chaincode 180–1
ChaincodeBuilder 175
chaincode deployment 185
ChaincodeGenerator class 175, 179
cloud-based analytics 224
cloud-based database service 168
cloud-based game clients 303–4
cloud-based systems 239
cloud computing (CC) 2, 17, 119, 263
Cloud Computing and Distributed Systems (CLOUDS) 28
Cloud-enabled Small Cells (CeSCs) 9
cloud gaming service operator 302
cloud RAN (CRAN) 55–6
cloud resources 106–7
cloud servers (CSs) 17, 263, 302
Cloud Service Providers (CSPs) 17
cloud-to-things (C2T) continuum 264
CoAP 78
ColorTracking 44
common public radio interface (CPRI) 66
communication protocols 23, 259–60
complex event processing (CEP) 78
Compound Annual Growth Rate (CAGR) 195
conceptual models 242–3
consensus algorithms 227, 278
consensus mechanisms 277
  analysis of algorithms 279
    energy efficiency 279–80
    network security 279–80
  energy trading scenario 280
    using PoA consensus algorithm 285
    using PoW consensus algorithm 282–5
  proof of authority 279
  proof of work 279
constant bitrate (CBR) 294
constrained variable bitrate (CVBR) 294
context-as-a-service (CooaS) 85
Context Query Engine (CQE) 85
Context Reasoning Engine (CRE) 85
Context Storage Management System (CSMS) 85
control plane 202
cost–benefit analysis 6
cost reduction 68
CouchDB 171
counter 204
Create, Read, Update, and Delete (CRUD) operations 145
credit-based threshold random walk (CB-TRW) 211–12
cryptographic techniques 277
Cumulocity IoT 86–7
customer premise networks (CPNs) 259
cyber-physical systems 266, 268
cybersecurity 265
  attacks in the field 266
    denial-of-service 266–7
    device compromise 267
    eavesdropping 266
    false data injection 266
  attacks on management 267
    APTS and insiders 267
    attacks on the cloud and edge 267–8
  defence mechanisms 268
    cloud and edge security 269
    intrusion detection 268
    IoT security 269

key principles 268
network security 268–9
Cygnus 83

data analysis 78
database services 117–18
data collection 78
data dependency 7
data plane 202
data pre-processing 78
data processing 232
data retention 111
data storage 78
decentralization 277
delegated Byzantine fault tolerance (DBFT) 279
delegated PoS (DPoS) 279
denial-of-Service (DoS) attacks 195, 228, 266–7
device-side agents 86
discontinuous reception (DRX) cycle 70
distributed DoS (DDoS) 266
distributed energy resources (DER) 256, 258
distributed physics 304–6
distributed virtual environments (DVEs) 302–3
distribution management systems (DMS) 257
distribution system operators (DSOs) 262
downlink control information (DCI) 57
Dynamic Host Configuration Protocol (DHCP) servers 203
dynamic resource discovery 26
dynamic scalability 26

eavesdropping 266
edge computing (EC) 119, 264–5, 268, 295, 304
edge resources 105–6
edge servers (ESs) 17
edge technologies 289–90
electrical power and energy systems (EPES) 261
electric vehicles (EVs) 258
elliptic curve cryptography (ECC) 269
encryption 231
energy management systems (EMS) 257, 261–2
energy storage systems (ESSs) 258
Engineering and Physical Sciences Research Council (EPSRC) 236
enhanced Mobile Broadband (eMBB) services 67
enhanced SLA manager approach 146
creation of SLA assets 146–50
deletion of SLA assets 152
reading SLA assets 150–1
updating SLA assets 151–2
estimateCost method 48
Ethereum 166, 170, 191
Ethereum blockchain network 172
Ethereum virtual machine (EVM) 134
European Telecommunications Standard Institute (ETSI) 80, 241
event of interest (EoI) 103
executor 21
Extensible Authentication Protocol (EAP) 201
Extensible Markup Language (XML) 134

fabric samples 185
FaceDetection 44
false data injection (FDI) 266
federated Byzantine agreement (FBA) 279
field area networks (FANs) 259
first-network 185
FIWARE 78, 83–4
flow hard timeout value 203
fog analytics 7
learning approaches for task offloading 8
reinforcement learning 10–12
supervised learning 8–10
unsupervised learning 8
task offloading using 12–13

FogBus2 framework 18
communication protocol 23
installation of 28
building from scratch 28–9
pulling from docker hub 29–30
interaction scenario 22–3
main capabilities 23
container-enabled 23–4
distributed multi-database platform support 27
dynamic resource discovery 26
dynamic scalability 26
multi-platform support 24
reusability 27
scheduling 24
supporting different topology models for communication 26–7
supporting heterogeneous IoT applications 27
usability 28
virtual private network support 27
main components 19–22
new IoT applications 35
evaluation results 49–50
implementation of 35–44
new scheduling policy 44–9
sample FogBus2 setup 30
P2P VPN setup 32–3
running FogBus2 components 33–5
fogbus2-ocr 30
fog computing 2, 18
architecture 2–5
task offloading 5
offload tasks 7
task and task offloading 5–7
fog nodes 11

GameOfLife Parallelized application 44
GameOfLife Pyramid application 44
GameOfLife Serialized application 44
general data protection regulation (GDPR) 225–6, 270
General Enablers (GE) 83
3rd Generation Partnership Project (3GPP) 54
geosynchronous equatorial orbit (GEO) 71
Google 292
graphical user interface (GUI) 134, 190
graphics processing units (GPUs) 289
grid architecture 256
functional domains 256–7
modern sub-systems 257–8
grid-to-vehicle (G2V) 258
group of pictures (GOP) 294
group wake up signaling 70
GTX 1080 296

Hadoop 114
Hadoop Open Source Implementation 116
Hash calculation 283
hash function 115
hashing algorithm 169
healthcare provider (HCP) 183
Health Insurance Portability and Accountability Act (HIPAA) 227
heuristic algorithm 60, 65
home area network (HAN) 259
home energy management systems (HEMSs) 259, 262
HTTP 80
human–computer interaction (HCI) 243
human–machine interfaces (HMIs) 262
hybrid automatic repeat request (HARQ) 71
hybrid cloud computing 17
hybrid model 307
Hyperledger Fabric (HLF) 137, 142, 166, 170–1, 180, 191
Hyperledger Sawtooth 191

immutability 277
InatelPlat 89
incremental algorithms 114
indestructibility 277

industrial area network (IAN) 259
Industrial Internet of Things (IIoT) 75, 248
industry leaders 87
InfluxDB 87
information technology (IT) 256
infrastructure-as-a-service (IaaS) 263
infrastructure resources 104
  cloud resources 106–7
    edge resources 105–6
    IoT devices 104–5
ingestion services 110–12
integer linear programming (ILP) problem 63
intelligent electronic devices (IEDs) 262
intelligent transport system (ITS) 11
interdependency 256
interest management 303
interference management 68
International Telecommunication Union's (ITU) 250
Internet of Medical Things (IoMT) 224
  application layer 225
  blockchain-based IoMT system 230
    application requirements 230–1
    multi-layered architecture 235–6
    system architecture 231–3
    system execution workflow 233–5
  connectivity layer 224–5
  GDPR compliance verification in 228
  general data protection regulation 225–6
  perception layer 224
  processing layer 225
Internet of Things (IoT) 1, 17, 53, 165, 167–8, 223, 239
  for developing quality-aware IoT applications 249
    conceptual framework to 250–1
  devices 104–5
  emulation 185–7
  human-in-the-loop *vs.* autonomic IoT applications 240–1
  key factors impacting quality in 245
    Quality of Context 248–9
    quality of data 247
    Quality of Device 248
    Quality of Information 249
    Quality of Processing 248
    quality of service 248
    Quality of User Interface 249
  quality 242
    in autonomic IoT applications 245
    in human-in-the-loop IoT applications 244–5
  quality of experience 241–2
Internet-of-Things Architecture (IoT-A) 81
Internet of Things (IoT) middleware platforms 75
  benchmarking 87
    metrics 88
    state-of-the-art in 89–91
  commercial IoT platforms 85
    AWS IoT 85–6
    Azure IoT 86
    Cumulocity IoT 86–7
    SmartFarmNet 87
    ThingWorx 87
  component and functionalities 81
    component and functionalities 81–2
    data processing and storage 82–3
    data retrieval and analysis 83
  holistic framework for benchmarking 91–2
  motivating scenario 77–9
  open source IoT middleware platforms 83
    Agri-IoT 85
    context-as-a-service 85
    FIWARE 83–4
    Kaa 84
    Node-RED 85
    OpenIoT 83
    ThingsBoard 84–5
    ThingSpeak 84
Internet Protocol (IP) 196

Internet Service Provider (ISP) 202
interoperability 256
intrusion detection and prevention system (IDPS) 196, 203–5
intrusion detection system (IDS) 205, 268
intrusion prevention system 196, 205
IoTBench 89
IRAFUTAL 135
  accessible by smart contracts 136
  amendable 136–7
  flexible 136
  identifiable and reusable 135–6
  loosely coupled 137
  tamper-proof 136
  user-friendly 136
ISO/IEC 19086-1 standard 130

Java library 190
Java programming language 191
Java programming library 167
JavaScript Object Notation (JSON) 205
  string 180
JavaScript programming language 186
JavaScript's asynchronous programming model 186
joint optimization algorithm 63–4

Kaa 84
Kaa sandbox 84
Kafka 111
Key Performance Indicators (KPIs) 76
K-nearest neighbors (KNN) algorithm 12
Konker 89

3-layered architecture model 80
learning module (LM) 12
LevelDB 171
lightweight edge anomaly detection algorithm 11
linear discriminant analysis (LDA) 117
link adaptation 70
Linksmart 89
local area network (LAN) 259
local logger 21
Local Network Control (LNC) unit 229, 231
long term evolution (LTE) 54
LoRAWaN 224
low-complexity algorithms 60
  Max-R 61
  Min-R 61
  Min-R relaxed and Max-R relaxed 61–2
low earth orbit (LEO) 71
Low Orbit Ion Cannon (LOIC) 213
low power wide area network (LPWAN) technology 54

machine learning (ML)
  algorithms 7, 75
  services 116–17
machine QoE (M-QoE) 245
machine-to-machine (M2M) communication 239
Man-in-the-Middle (MitM) attacks 195
MapReduce model 114, 116
master telemetry unit (MTU) 262
Max-R algorithm 61
mean opinion scores (MOS) 244
media access control (MAC) 196
Micro Benchmarks 88
micro-modular data centres (MMDCs) 265
Microsoft 292
Microsoft Azure 78
Microsoft SQL Server 87
middleware layer 80
Min-R algorithm 61
Mirai 267
MongoDB 83
MQTT 78, 80
Multi-layered architecture 235–6
Mutual Authentication (MA) 233

naive Bayes 117
NaiveFormula Parallelized 44
NaiveFormula Serialized 44
naive SLA manager approach 145–6

narrowband-Internet of Things (NB-IoT) 53, 224
  advantages 68
    cost reduction 68
    interference management 68
    optimal BBU resource utilization 68
    spectrum utilization 68
  with CRAN 67–8
  deployment modes of 54
  physical layer architecture 56–8
  resource management in 59
    allocation algorithms 59–66
    performance metrics 59
  standardization 68
    Release 16 68–70
    Release 17 70–1
narrowband physical broadcast channel (NPBCH) 57
narrowband physical downlink control channel (NPDCCH) 57
narrowband physical downlink shared channel (NPDSCH) 57
natural language processing (NLP) 75
neighbour discovery protocol (NDP) 218
neighbourhood area network (NAN) 259
Netflix 292
network architecture 258
  area networks 258–9
  communication protocols 259–61
networking services 109–10, 119
network security 268–9, 279–80
new radio (NR) technologies 54
NodeJS 84, 186, 190
Node-RED 85
Non-dominated Sorting Genetic Algorithm 2 (NSGA2) 21
Non-dominated Sorting Genetic Algorithm 3 (NSGA3) 21
non-terrestrial networks (NTN) 70–1
Northbound Interfaces (NBI) 197
NumericalOperator 175–6
NVIDIA 292

OpenFlow protocol 198–9
OpenIoT 78, 83
Open Systems Interconnection (OSI) model 201
Open Virtual Switch (OvS) 203
OpenVSwitch (OvS) 206
operational expenditures (OPEX) 55
operational technology (OT) 256
OPNET simulator 227
optimal BBU resource utilization 68
optimized allocation 62–3
Optimized History-based Non-dominated Sorting Genetic Algorithm (OHNSGA) 21
Orion+STH 89
orthogonal frequency division multiplexing (OFDM) 54

P2P VPN setup 32–3
packet 203
PacketIn event 203
packets per second (pps) 213
performance metrics 59
  power consumption 59
  resource utilization 59
personal area networks (PANs) 260
phasor data concentrators (PDCs) 263
phasor measurement units (PMUs) 263, 271
physical unclonable function (PUF) 269
physical uplink control channel (PUCCH) 58
physical uplink shared channel (PUSCH) 58
piconets 260
plant EMS (PEMS) 262
platform-as-a-service (PaaS) 263
point of common coupling (PCC) 258
Port Bingo (PB) algorithm 211
port-scanning technique 195, 196
PostgreSQL 87
power boosting 69–70
power line communication (PLC) 260, 271

practical Byzantine fault tolerance (PBFT) 279
preparatory experiments 212
  characteristics of attacks 213–15
  IDPS algorithm threshold settings 215–17
primary fog node 6
programmable logic controllers (PLCs) 262
proof of authority (PoA) 278–9
  consensus algorithm 285
  number of transactions *vs.* time to complete mining process for 284
  validation process in 284
proof of elapsed time (PoEt) 278
proof of stake (PoS) 232, 278
proof of work (PoW) 232, 278–9
  mining process in 282
  number of transactions *vs.* nonce value for 283
PTC 87
Public Network Control (PNC) unit 229
Python 84

quadrature amplitude modulation (QAM) 71
Quality of Context (QoC) 248–9
quality of data (QoD) 246–7
Quality of Device (QoDe) 247–8
Quality of Experience (QoE) 239, 242, 301
  conceptual models 242–3
  ML/AI-based models 243
  statistical models 243
Quality of Information (QoI) 247, 249
Quality of IoT (QoIoT) 245
Quality of Machine Experience (QoME) 245
Quality of Processing (QoP) 246, 248
quality of service (QoS) 18, 55, 165, 211–12, 246, 248, 295
Quality of Things (QoT) 245, 249
Quality of User Interface (QoU) 247, 249
quantisation parameter (QP) 293
query processing 83

RabbitMQ 112
radio access network (RAN) 55
rankApplicationTasks 47
Raspberry Pi 85, 235
rate limiting (RL) 211
RDF Stream Processing (RSP) 85
real-time analysis services 186
real-time interaction game 300–1
real-time strategy games (RTS) 300
recomputation algorithms 114
reinforcement learning 10–12
Relational Database Service (RDS) 82
Release 16 68
  group wake up signaling 70
  link adaptation 70
  multiple grants within single DCI 68–9
  power boosting 69–70
  resource block alignment 69
  uplink resources 70
Release 17 70
  higher peak data rates 70
  support of NB-IoT over non-terrestrial networks (NTN) 70
Remote Authentication Dial-In User Service (RADIUS) 201
remote health-monitoring service (RHMS) 98
remote logger 20
Remote Patient Monitoring (RPM) 181
remote radio head (RRH) 55
remote telemetry units (RTUs) 262, 271
renewable energy sources (RES) 258
resource block alignment 69
resource management techniques 18
reusability 27
round-trip-time (RTT) 202
RTX 2060 296
Ruby 84
running FogBus2 components 33–5

secondary fog nodes 2, 6
SENSEI project 80
sensing services 109
sensors 76
server-side agents 86
service level agreement (SLA) 127, 165, 168–9, 173–5, 183–4
  agnostic approaches 130
    implicit SLA representation 131
    zero representation within blockchain 131
  aware approaches 132
    automated smart contract generation 134
    deployment to compatible decentralised storage 132
    manual smart contract development 132–4
  data model 143
    enhanced SLA manager approach 146–52
    naive SLA manager approach 145–6
  definition and negotiation 129
  end-to-end SLA conceptual model for IoT applications 100–4
  evaluation 119
    experiment 119
    experimental results 121
    participants 119
    procedure 119–20
  evaluation and observation 152
    definition 153–4
    failure test units 152–3
    negotiation 154–8
    proposed approach *vs.* conventional approaches 158
    threats to validity 158
  FromSLAToSmartContract library 174
    chaincode 180–1
    generating Chaincode from Java SLA 178–80
    parsing JSON SLA into a Java SLA 176–8
  infrastructure resources 104
    cloud resources 106–7
    edge resources 105–6
    IoT devices 104–5
  IRAFUTAL 135
    accessible by smart contracts 136
    amendable 136–7
    flexible 136
    identifiable and reusable 135–6
    loosely coupled 137
    tamper-proof 136
    user-friendly 136
  negotiation in related works 134–5
  proposed SLA representation approach 137
    blockchain assets 138–9
    formal SLA specification 139–40
    state storage capability 137–8
  service concept 107
    batch processing services 114–16
    database services 117–18
    ingestion services 110–12
    machine-learning services 116–17
    networking services 109–10
    sensing services 109
    stream processing services 112–14
  to smart contract Java library 174
  violation 168
Service Level Agreement VErified (SLAVE) 171
service level objectives (SLOs) 129, 173, 175
Service Vehicles (SaV) 10
signal to interference plus noise ratio (SINR) 64
SLA2C 134
SLAC 134, 172
Small Cell Cloud (SCC) 9
smart contracts 127–8, 136, 166, 169–70, 180, 184, 232
smart farming 75–6
SmartFarmNet 76, 87
smart gateways 105

smart grid 2
  cybersecurity 265
    attacks in the field 266–7
    attacks on management 267–8
    defence mechanisms 268–9
  functional domains of 257
  grid management
    using cloud 263–4
    using edge computing 264–5
  infrastructure 255
    grid architecture 256–8
    key concepts 256
    network architecture 258–61
  IoT-based communication protocols used in 259
  management systems 261
    EMS 261–2
    SCADA 262
    WAMS 263
smart meters 257
smart patient monitoring system 167
software-as-a-service (SaaS) 263
Software-Defined Compute (SDCompute) 200
Software Defined Data Centre (SDDC) 199
software-defined network (SDN) 8, 195
  attack experiments and results 217–18
  components 197
    applications 197
    CDPI 198
    controller 197
    datapath 198
    interface drivers and agents 198
    management and admin 198
    Northbound interfaces 198
  controller 198–9
  conventional networking *vs.* 196–7
  detecting ARP spoofing attacks in 202
    ARP spoofing 206
    blacklisted MAC address 206–8
    detection technique 203
    IDPS 203–5
    standard and spoofed ARP requests 210–11
    testbed 205–6
  detecting port-scanning and DoS attacks in 211
    characteristics of attacks 213–15
    credit-based threshold random walk 212
    IDPS algorithm threshold settings 215–17
    quality of service 212
    rate limiting 211
  OpenFlow protocol 199
  SDN-based cloud 199–201
  security 201
    application plane 201–2
    control plane 202
    data plane 202
Software-Defined Parameter (SDP) 201
Software-Defined Security (SDSec) 200
Software-Defined Storage (SDStorage) 200
Software-Defined Systems (SDSys) 199–200
spectrum utilization 68
statistical models 243
streamed gaming 289
  multiplayer streamed gaming 302
    cloud-based game clients 303–4
    distributed physics 304–6
    distributed virtual environments 302–3
    interest management 303
  streaming video games 291
    experience 301
    gaming requirements 295–6
    genre and timeliness 298–301
    handling legacy titles 297–8
    origins 291–2
    platform expectations 296–7
    streamed media 292–5
    summary 301–2

streamed media 292–5
stream processing services 112–14
Subjective Quality Assessments (SQAs) 244
supervised learning 8–10
supervisory control and data acquisition (SCADA) 257, 262–3, 271
supply chain management 277
Sybil attack 280
synchronisation signal block (SSB) 70

task executor 21–2
task offloading 2
Task Vehicles (TaV) 10
Tcpreplay 213
testbed 205–6
ThingsBoard 84–5
ThingSpeak 84
ThingWorx 87
three-layered fog-computing architecture 3
topic detection and tracking (TDT) 117
TopoGuard 202
transmission domain 263
transmission layer 80
transmission management systems (TMS) 257
transmission system operators (TSOs) 262
transmission time interval (TTI) 54
Transport Layer Security (TLS) protocol 201, 233
trusted-third party (TTP) 166, 171
turn-based game 299
two-way authentication 233

ultra reliable low latency communications (URLLC) services 67
Union ITU-T (ITU-T) 241
unsupervised learning 8
update methods 179
uplink resources 70
user-defined map function 115

variable bitrate (VBR) 294
vehicle-to-grid (V2G) 258
VideoOCR algorithm 35, 44
virtual private networks (VPNs) 27, 269

WebSocket protocols 233
wide area monitoring system (WAMS) 257, 263, 271
wide area network (WAN) 259
WiFi 224
WiMax 105
wireless LANs (WLANs) 261

Zero-Trust 201
Zigbee 105, 260
Z-Wave 261

Managing Internet of Things Applications across Edge and Cloud Data Centres

Cloud computing has been a game changer for internet-based applications such as content delivery networks, social networking and multi-tier enterprise applications. However, the requirements for low-latency data access, security, bandwidth, mobility, and cost have challenged centralized data center-based cloud computing models, which is driving the need for the novel computing paradigms of edge and fog computing. The internet of things (IoT) focuses on discovery, aggregation, management, and acting on data originating from internet-connected devices via programmable sensors, actuators, mobile phones, surveillance cameras, routers, gateways and switches. But the aggregation of this data is expensive and can be time consuming.

Traditional cloud-centric resource management models need to move towards more distributed and decentralized models so that they can cope with the challenges posed by the evolution of IoT smart devices and network solutions. However, supporting IoT data processing across cloud and edge data centers is not a trivial challenge. IoT sensing devices must be configured as a collection of data-analytics driven workflows where each node in the process can essentially run on multiple heterogeneous cloud and edge data centers.

This book presents state-of-the-art interdisciplinary computing research in the application lifecycle management for internet of things in edge and cloud computing. The book addresses challenges from a distributed system perspective that includes both cyber and physical aspects. The authors aim to bring together the four paradigms of cloud and edge computing, cyber physical systems, internet of things and big data for future ICT systems.

Written and edited by an international team of experts in the field, this book offers key insights to researchers, engineers, IT professionals, advanced students, postgraduate students and lecturers working in the fields of parallel and distributed computing, data mining, information retrieval, c cloud, edge and fog computing, and the IoT.

About the Editors

Rajiv Ranjan is university chair professor for internet of things research, co-director of the Networked and Ubiquitous Systems Engineering (NUSE) Group and academic director of the School of Computing, as well as research director of the Newcastle Urban Observatory at Newcastle University, UK.

Karan Mitra is an associate professor in the Pervasive and Mobile Computing Group in the Department of Computer Science, Electrical and Space Engineering at Luleå University of Technology, Sweden.

Prem Prakash Jayaraman is a professor in internet of things and distributed systems at Swinburne University of Technology, Australia. He is also the Director of Factory of the Future, and the Digital Innovation Lab at Swinburne

Albert Y. Zomaya is Peter Nicol Russell chair professor of computer science and director of the Centre for Distributed and High-Performance Computing in the School of Computer Science at the University of Sydney, Australia.

The IET Book Series on **Big Data**

Editor-in-Chief: *Albert Y. Zomaya, University of Sydney, Australia*

This book series explores the myriad of research activities that analyze, compute, store, manage, and transport big data, such as algorithm design, data mining and search, processor architectures, databases, infrastructure development, service and data discovery, networking and mobile computing, edge, fog and cloud computing, high-performance computing, privacy and security, storage, and visualization.

ISBN 978-1-78561-779-9

9 781785 617799 >

The Institution of Engineering and Technology
theiet.org
978-1-78561-779-9